Library Management System

Mini Project Presentation

Subject: Java For Non-Tech

Topic: OOP Implementation

Student Name: **Sonu Sagar**

RID: **R54818**

Branch: **Computer Science**

Sem/Year: **6th/3rd**

Introduction

- Library Management System is a console- based Java application.
- It manages books in a library.
- Implements Object-Oriented Programming concepts.

Advantages

- Easy to use
- Simple console interface
- Demonstrates real-time application of OOP

Objectives

- To manage book records
- To implement menu-driven program
- To issue and return books
- To demonstrate OOP concepts

OOP Concepts Used

- Class & Object
- Encapsulation
- Constructor
- ArrayList Collection
- Methods and Control Structures

Book & Library Class

- Attributes:
- - bookId
- - title
- - author
- - isIssued
- Methods: issueBook(), returnBook(), displayBook()

- Stores list of books using ArrayList
- addBook()
- displayBooks()
- findBook()

Features

- Add new books
- Display available books
- Issue book
- Return book
- Exit system

Sample Output

- ===== Library Management System =====
- 1. Add Book
- 2. Display Books
- 3. Issue Book
- 4. Return Book
- 5. Exit

```java
// Constructor
public Book(int bookId, String title, String author) {
    this.bookId = bookId;
    this.title = title;
    this.author = author;
    this.isIssued = false;
}
```

```java
import java.util.ArrayList;
import java.util.Scanner;

// Book Class
class Book {
    private int bookId;
    private String title;
    private String author;
    private boolean isIssued;
```

1

```java
    // Getters
    public int getBookId() {
        return bookId;
    }

    public String getTitle() {
        return title;
    }

    public String getAuthor() {
        return author;
    }

    public boolean isIssued() {
        return isIssued;
    }
```

2

```java
    // Issue book
    public void issueBook() {
        if (!isIssued) {
            isIssued = true;
            System.out.println("Book issued successfully!");
        } else {
            System.out.println("Book is already issued.");
        }
    }
```

3

```java
    // Return book
    public void returnBook() {
        if (isIssued) {
            isIssued = false;
            System.out.println("Book returned successfully!");
        } else {
            System.out.println("Book was not issued.");
        }
    }

    // Display book details
    public void displayBook() {
        System.out.println("ID: " + bookId +
                " | Title: " + title +
                " | Author: " + author +
                " | Status: " + (isIssued ? "Issued" : "Available"));
    }
}
```

4

```java
// Library Class
class Library {
    private ArrayList<Book> books = new ArrayList<>();

    // Add book
    public void addBook(Book book) {
        books.add(book);
        System.out.println("Book added successfully!");
    }

    // Display all books
    public void displayBooks() {
        if (books.isEmpty()) {
            System.out.println("No books in library.");
            return;
        }
        for (Book book : books) {
            book.displayBook();
        }
    }

    // Search book by ID
    public Book findBook(int bookId) {
        for (Book book : books) {
            if (book.getBookId() == bookId) {
                return book;
            }
        }
        return null;
    }
}
```

5

```java
// Main Class
public class LibraryManagementSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Library library = new Library();

        int choice;

        do {
            System.out.println("\n===== Library Management System =====");
            System.out.println("1. Add Book");
            System.out.println("2. Display Books");
            System.out.println("3. Issue Book");
            System.out.println("4. Return Book");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();
```

6

```java
switch (choice) {
    case 1:
        System.out.print("Enter Book ID: ");
        int id = sc.nextInt();
        sc.nextLine(); // clear buffer
        System.out.print("Enter Title: ");
        String title = sc.nextLine();
        System.out.print("Enter Author: ");
        String author = sc.nextLine();

        Book newBook = new Book(id, title, author);
        library.addBook(newBook);
        break;

    case 2:
        library.displayBooks();
        break;

    case 3:
        System.out.print("Enter Book ID to issue: ");
        int issueId = sc.nextInt();
        Book bookToIssue = library.findBook(issueId);
        if (bookToIssue != null) {
            bookToIssue.issueBook();
        } else {
            System.out.println("Book not found.");
        }
        break;
```

7

```java
            case 4:
                System.out.print("Enter Book ID to return: ");
                int returnId = sc.nextInt();
                Book bookToReturn = library.findBook(returnId);
                if (bookToReturn != null) {
                    bookToReturn.returnBook();
                } else {
                    System.out.println("Book not found.");
                }
                break;

            case 5:
                System.out.println("Exiting system... Thank you!");
                break;

            default:
                System.out.println("Invalid choice! Try again.");
        }

    } while (choice != 5);

    sc.close();
  }
}
```

output

```
===== Library Management System =====
1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Exit
Enter your choice: 1
Enter Book ID: 101
Enter Title: Java Programming
Enter Author: James Gosling
Book added successfully!

Enter your choice: 2
ID: 101 | Title: Java Programming | Author: James Gosling | Status: Available

Enter your choice: 3
Enter Book ID to issue: 101
Book issued successfully!
```

Conclusion

- Library Management System successfully demonstrates core Java OOP concepts in a practical way.

Future Enhancements

- Add database connectivity (MySQL)
- Add login authentication
- Add fine calculation